

SO(3) Equivariant Neural Networks

Lecture 3

How Geometric Types Combine

Tensor products and Clebsch–Gordan coefficients

90-minute journal-club lecture note

Phong Dang • prepared with AI assistance

Companion notebook included in the series package

Session map and learning goals

hook	tensor products	$1 \otimes 1$	break	CG in AI	notebook	discussion
0			45			90 min

By the end of this lecture, you should be able to:

- interpret a tensor product as the space of pairwise feature products;
- use the triangle rule to list allowed output irreps;
- explain $1 \otimes 1 = 0 \oplus 1 \oplus 2$ using dot, cross, and quadrupole products;
- describe CG coefficients as a fixed change-of-basis table;
- state and numerically test the CG intertwining identity;
- construct $D^{(\ell)}(R)$ for any ℓ by coupling.

Key idea

Clebsch–Gordan coefficients are not mysterious learned constants. They are fixed coordinates for sorting a product of two rotating objects into pieces that each have a definite transformation type.

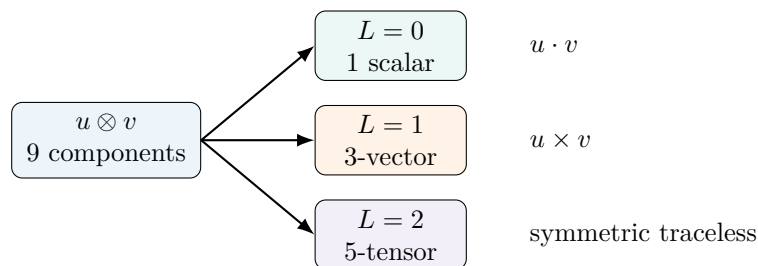
1 Opening puzzle: where do nine products go?

Two vectors have components u_i and v_j . Their outer product has nine entries $u_i v_j$. Representation theory says these nine degrees of freedom can be reorganized as

$$3 \times 3 = 1 + 3 + 5, \quad (1)$$

or, using irrep labels,

$$1 \otimes 1 = 0 \oplus 1 \oplus 2. \quad (2)$$



The dimensions are not a coincidence. These are precisely the irreducible rotational pieces of the outer product.

2 Tensor products and the triangle rule

If x transforms in irrep ℓ_1 and y in irrep ℓ_2 , their products transform in the tensor product representation:

$$x \otimes y \mapsto \left(D^{(\ell_1)}(R) \otimes D^{(\ell_2)}(R) \right) (x \otimes y). \quad (3)$$

For $\text{SO}(3)$, this product decomposes as

$$\ell_1 \otimes \ell_2 = \bigoplus_{L=|\ell_1-\ell_2|}^{\ell_1+\ell_2} L. \quad (4)$$

Therefore the allowed outputs satisfy

$$|\ell_1 - \ell_2| \leq L \leq \ell_1 + \ell_2. \quad (5)$$

Examples:

$$0 \otimes 2 = 2, \quad (6)$$

$$1 \otimes 1 = 0 \oplus 1 \oplus 2, \quad (7)$$

$$1 \otimes 2 = 1 \oplus 2 \oplus 3, \quad (8)$$

$$2 \otimes 2 = 0 \oplus 1 \oplus 2 \oplus 3 \oplus 4. \quad (9)$$

The dimensions provide a useful check. For example,

$$(2 \cdot 1 + 1)(2 \cdot 2 + 1) = 3 \cdot 5 = 15 = 3 + 5 + 7. \quad (10)$$

Key idea

One caution about the code. The notebook sets `L_MAX = 2` and builds only couplings with $\ell \leq 2$, so $1 \otimes 2 \rightarrow 3$ and $2 \otimes 2 \rightarrow 3, 4$ never appear in its tables. That truncation is a *budget* decision, not a consequence of the triangle rule. The rule above is exact; the code is a deliberate subset of it, and raising `L_MAX` restores the missing paths at rising cost.

Math lens

For integer angular momenta in $\text{SO}(3)$, each allowed L appears once in $\ell_1 \otimes \ell_2$. This multiplicity-free property keeps the coupling notation simple. Neural networks still have channel multiplicities because we deliberately carry several copies of each irrep.

3 The three pieces of two vectors

The scalar piece is proportional to

$$s = u \cdot v. \quad (11)$$

The antisymmetric vector piece is proportional to

$$a = u \times v. \quad (12)$$

The symmetric-traceless piece can be written

$$Q = \frac{1}{2}(uv^\top + vu^\top) - \frac{1}{3}(u \cdot v)I. \quad (13)$$

It is symmetric and has $\text{tr } Q = 0$, leaving five independent components.

Physics lens

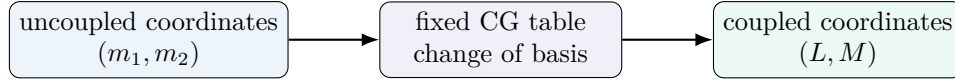
The decomposition separates isotropic correlation ($L = 0$), handed or rotational response ($L = 1$), and quadrupolar alignment ($L = 2$). These patterns recur in angular momentum, molecular multipoles, continuum mechanics, and atomistic machine learning.

4 Clebsch–Gordan coefficients

The uncoupled product basis is labeled by (m_1, m_2) . The coupled basis is labeled by (L, M) . Clebsch–Gordan coefficients are the change-of-basis entries:

$$z_M^{(L)} = \sum_{m_1, m_2} \underbrace{\langle \ell_1 m_1, \ell_2 m_2 | LM \rangle}_{C_{\ell_1 m_1, \ell_2 m_2}^{LM}} x_{m_1}^{(\ell_1)} y_{m_2}^{(\ell_2)}. \quad (14)$$

The magnetic selection rule $M = m_1 + m_2$ makes most entries zero.



The notebook loads coefficients from `sympy.physics.wigner.clebsch_gordan`; it does not derive or fit them. It attaches a fixed phase $(-i)^{\ell_1 + \ell_2 - L}$ to match the chosen reality convention. Changing a whole irrep basis by a consistent phase changes coordinates, not physics.

5 Building $D^{(\ell)}$ for any ℓ

Lecture 2 gave the Wigner matrices for the two smallest types,

$$D^{(0)}(R) = 1, \quad D^{(1)}(R) = U_1 R U_1^\dagger, \quad (15)$$

and left the general case open, because the missing ingredient is the table we have just built.

Those two already determine every other ℓ . Every irrep of $\text{SO}(3)$ is contained in repeated tensor products of the vector representation, and the triangle rule applied to $(\ell - 1) \otimes 1$ gives

$$(\ell - 1) \otimes 1 = (\ell - 2) \oplus (\ell - 1) \oplus \ell, \quad (16)$$

so ℓ appears there exactly once. The $(2\ell + 1)$ -dimensional subspace carrying it is therefore unique, and the coefficients that project onto it are the Clebsch–Gordan coefficients of Equation (14) with $\ell_1 = \ell - 1$, $\ell_2 = 1$ and $L = \ell$:

$$C_{M;ab}^{(\ell)} = \langle \ell - 1, a; 1, b | \ell, M \rangle. \quad (17)$$

Rotating the product $D^{(\ell-1)} \otimes D^{(1)}$ and projecting back onto that subspace gives a recursion that reaches every ℓ .

Math lens

Starting from $D^{(1)}(R) = U_1 R U_1^\dagger$, every higher Wigner matrix follows from

$$D_{MN}^{(\ell)}(R) = \sum_{a,b,i,j} C_{M;ab}^{(\ell)} D_{ai}^{(\ell-1)}(R) D_{bj}^{(1)}(R) (C_{N;ij}^{(\ell)})^*, \quad \ell \geq 2. \quad (18)$$

Nothing here is specific to $\ell = 2$: rotate the product of a smaller multiplet and a vector, then project back onto the part that transforms as ℓ . Iterating Equation (18) to $\ell = 4$ reproduces unitarity and the representation law $D(R_1)D(R_2) = D(R_1 R_2)$ to roughly 10^{-15} .

The next section says in one line why this works, so it is worth reading Equation (18) again after it.

The notebook implements the $\ell = 2$ case, written with the $1 \otimes 1 \rightarrow 2$ tensor because $\ell - 1 = 1$ there. Raising `L_MAX` means iterating the same line, not writing new mathematics.

A closed form also exists. In terms of Euler angles $R = R_z(\alpha)R_y(\beta)R_z(\gamma)$,

$$D_{mm'}^{(\ell)} = e^{-im\alpha} d_{mm'}^{(\ell)}(\beta) e^{-im'\gamma}, \quad (19)$$

where the real *little-d* matrix is given by a finite sum of factorials. This is what a library routine evaluates, and it is the efficient choice at large ℓ . We prefer Equation (18) in this course because it needs no machinery beyond the coupling tables we need anyway, and because it makes the structural point visible: high- ℓ features are built out of vectors, exactly as an equivariant layer builds them.

Key idea

Any of these routes gives a valid representation, but not necessarily the *same matrices*. Phase and axis conventions differ between sources. With the spherical basis U_1 fixed in Lecture 2, a rotation by θ about $\hat{n}$ gives

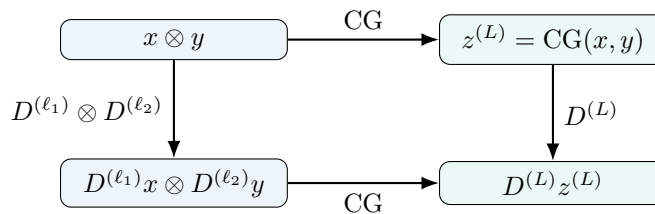
$$D^{(\ell)}(R) = \left[e^{-i\theta \hat{n} \cdot \mathbf{J}^{(\ell)}} \right]^*, \quad (20)$$

the complex *conjugate* of the form quoted in most quantum mechanics texts. Mixing a library's $D^{(\ell)}$ with this basis silently breaks equivariance. Before trusting any implementation, run the two checks the notebook runs: $D(R_1)D(R_2) = D(R_1 R_2)$, and agreement with the Cartesian route at $\ell = 1$.

6 The intertwining identity

The essential property is

$$\text{CG}\left(D^{(\ell_1)}(R)x, D^{(\ell_2)}(R)y\right) = D^{(L)}(R)\text{CG}(x, y). \quad (21)$$



A map satisfying this equation is called an *intertwiner*. This is the exact reason a CG coupling can be placed inside an equivariant neural network.

It is also the one-line reason Equation (18) works. Written as matrices, Equation (21) says $C(D^{(\ell_1)} \otimes D^{(\ell_2)}) = D^{(L)}C$, and the CG table is an isometry onto the coupled subspace, so $CC^\dagger = I$. Multiplying on the right by $C^\dagger$ therefore gives $C(D^{(\ell_1)} \otimes D^{(\ell_2)})C^\dagger = D^{(L)}$, which is exactly the recursion with $\ell_1 = \ell - 1$ and $\ell_2 = 1$.

AI lens

The CG tensor controls the m indices and is fixed. A neural network adds learnable weights over channel indices. This preserves the geometric coupling rule while allowing task-dependent combinations of many feature channels.

7 The coupling is already a nonlinearity

Look again at Equation (14). It is *bilinear*: linear in x for fixed y , linear in y for fixed x , and therefore *quadratic* in the pair. When both arguments are features of the same sample, the output is a quadratic function of the input.

That single observation separates an equivariant network from an ordinary one. Stack linear layers with no activation and they collapse: a product of matrices is a matrix, so depth buys nothing. This is the entire reason a multilayer perceptron needs tanh or ReLU between its layers. A stack of CG layers does not collapse. Each tensor product multiplies two features together, so the polynomial degree the network can reach doubles with every coupling layer.

For the architecture of Lecture 4 the bookkeeping is short. Inputs are degree 1. The first layer's tensor products give degree 2. The second layer couples two degree-2 features and reaches degree 4. The same- ℓ channel-mixing paths are degree-preserving; only the coupling paths raise it.

AI lens

An equivariant network is nonlinear *before* any activation function is added. That is not true of an MLP, and it is the most commonly missed structural difference between the two.

The nonlinearity you get this way is polynomial, and at fixed depth the degree is bounded. Anything that saturates, thresholds, or decays exponentially in an invariant has to come from somewhere else — which is exactly the job of the scalar gates in Lecture 4. Lecture 4's notebook measures both effects directly by reading polynomial degree off a log-log slope.

8 Reading the notebook contraction

The function computes

```
np.einsum('Mij,...i,...j->...M', C, x, y)
```

where $i = m_1$, $j = m_2$, and M is the output magnetic index. The ellipsis represents any shared batch or channel axes. The output has $2L + 1$ components.

The notebook checks Equation (21) using arbitrary complex x and y . These are test vectors in the irrep spaces, not learned parameters. Generic complex vectors test the full complex linear identity; physical real tensors form a constrained subset.

9 Notebook laboratory

Open `lecture_03_clebsch_gordan.ipynb`. You will:

1. load CG coefficients from SymPy;
2. inspect nonzero entries and selection rules;
3. couple two physical vectors and recover $(u \cdot v)/\sqrt{3}$ and $(u \times v)/\sqrt{2}$ exactly;
4. verify the CG intertwining identity;
5. replace one CG tensor with a random tensor and observe failure.

The two normalizations in item 3 are worth stating plainly, because they are often left as an empirical fit. With the spherical components and the $(-i)^{\ell_1+\ell_2-L}$ phase used here, the couplings of two vectors are exactly

$$z^{(0)} = \frac{u \cdot v}{\sqrt{3}}, \quad z^{(1)}|_{\text{Cartesian}} = \frac{u \times v}{\sqrt{2}}, \quad (22)$$

to machine precision. The factors come from the CG normalization itself; nothing has to be calibrated.

10 Questions and discussion

Question 3.1. Use the triangle rule to decompose $0 \otimes 2$, $1 \otimes 2$, and $2 \otimes 2$. Check the dimensions in each case.

Question 3.2. Explain $1 \otimes 1 = 0 \oplus 1 \oplus 2$ using familiar Cartesian operations.

Question 3.3. In what sense are CG coefficients a change of basis? Name the old and new labels.

Question 3.4. Are CG coefficients learned by the network? If not, where does learnable flexibility enter a CG layer?

Question 3.5. What two computational routes are compared in the intertwining test?

Question 3.6. Why does replacing a CG tensor with a random tensor of identical shape usually break equivariance?

Question 3.7. The network of Lecture 4 takes $\ell = 1$ inputs and applies two coupling layers. With no activation function anywhere, what is the highest polynomial degree in the inputs that it can produce, and which path achieves it? Which paths cannot raise the degree at all?

Checkpoint

You are ready for Lecture 4 if you can say: CG tensors determine which m components combine into an output type L ; learned weights decide how strongly the resulting channel paths contribute.